

Enterprise Document Intelligence Platform

Project Brief

Assigned to: Joshika Dyavanapelli

1. Overview

You're building a production-shaped Enterprise Document Intelligence Platform: a multi-tenant, role-gated system where users upload documents (including scanned, non-text PDFs and images), ask questions in a chat interface, and get grounded answers with citations, the same RAG discipline as your first project, but now embedded inside a real application with authentication, authorization, async processing, caching, observability, and a deployable infrastructure story.

This is not a bigger version of Project 1. It's a different kind of exercise: Project 1 tested whether you understand retrieval. This one tests whether you can design and reason about a full system where retrieval is just one component among many, and where the pieces have to work together correctly under real constraints, concurrent users, slow uploads, cache invalidation, failing dependencies, permission boundaries.

No RAG frameworks (LangChain, LlamaIndex, etc.) same rule as before. No shortcuts on auth either: no third-party auth-as-a-service. Build the primitives yourself so the reasoning is visible.

2. Required Technology Stack

The stack below is mandatory, not a menu. Substitutions are not permitted without prior sign-off.

Layer	Required Technologies
Frontend	React 18 + Vite, TypeScript, TanStack Query, Zustand, React Router, Tailwind CSS, Vitest + React Testing Library, Playwright.
API / Backend	Node.js + Express (or NestJS) in TypeScript for the core API and auth; Python + FastAPI as a dedicated microservice for OCR, chunking, embeddings, and the RAG/NLP pipeline. The two services communicate over REST or gRPC, no monolith.
Auth & Security	JSON Web Tokens (access + refresh), bcrypt or argon2 for password hashing, role-based middleware/guards, rate limiting (e.g. express-rate-limit or Nginx-level).
Queueing	RabbitMQ (or BullMQ on Redis) for async document processing jobs, OCR, chunking, embedding.
Caching	Redis: session/token cache, query-result cache, embedding cache.
Database	MongoDB for users, documents, chat threads, and analytics/audit events.
Vector Search	Self-managed FAISS service (own HTTP wrapper around it) — no managed vector DB, consistent with building retrieval yourself.
OCR	Tesseract (pytesseract) for scanned PDFs and images.
Embeddings & NLP	sentence-transformers for embeddings; spaCy for entity/intent extraction feeding the NLP bot layer on top of the RAG core.
LLM	Ollama running a local open-source model (e.g. llama3), consistent with Project 1.
Containers & Orchestration	Docker + Docker Compose for local dev; Kubernetes manifests or a Helm chart for Deployments, Services, Pods, and a HorizontalPodAutoscaler.
Edge / Networking	Nginx as reverse proxy/ingress (TLS termination, routing, rate limiting); ingress-nginx if running on Kubernetes; a documented DNS/service-discovery model.
CI/CD	GitHub Actions: lint, unit tests, build, and a stage for integration/load tests.
Testing	Vitest + React Testing Library (frontend unit), Jest or Pytest (backend unit), Supertest (API integration), Playwright (e2e), k6 (load/latency).
Observability	Structured JSON logging (pino or Winston) with request-ID tracing across services, Prometheus for metrics, Grafana for dashboards.

3. Product Requirements

3.1 Core user flow

- A user signs up, verifies, and logs in. Their role (Admin / Editor / Viewer) determines what they can do.
- A user uploads a document, text-native PDF, scanned/image-based PDF, or a raw image. Scanned/image content is run through OCR before entering the RAG pipeline.
- Documents are chunked, embedded, and indexed asynchronously (not blocking the request) with visible processing status.
- A user asks questions in a persistent chat thread per document. Answers are grounded strictly in the document, cite page/section, and explicitly say when the answer isn't present.
- An Admin can see usage analytics: query volume, latency percentiles, failure rate, per-user/per-document activity.

3.2 Multi-tenancy & RBAC

- Documents and chat threads are scoped to the user/org that created them. No cross-tenant data leakage, enforced at the query layer, not just the UI.
- Three roles minimum: Admin (manage users, view analytics, full document access), Editor (upload, query, delete own documents), Viewer (query only, read-only).
- Permission checks happen server-side on every relevant endpoint. Hiding a button in the UI is not access control.

4. Frontend Requirements (React + Vite)

4.1 Authentication & authorization

- Signup/login/logout with client-side validation and clear error states (wrong password, expired session, account locked, etc.).
- JWT access token + refresh token flow, with silent refresh and forced logout on refresh failure.
- Passwords hashed server-side (bcrypt or argon2), the frontend's job is safe transport (HTTPS-only, no logging of raw credentials, no plaintext in localStorage).
- Route guarding by role: unauthorized routes redirect, not just hide; deep-linking to a forbidden page must not flash protected content before redirecting.

4.2 Core application UX

- Chat interface with streaming responses (SSE or WebSocket), answers should appear incrementally, not as one blocking response.
- Upload flow with progress indication, drag-and-drop, per-file processing status (queued → OCR → chunking → embedding → ready), and a clear failure state per file.
- Global and per-request error handling: network failure, 401/403, 5xx, timeout, each should produce a distinct, honest message, not a generic "something went wrong."
- Loading states that reflect real state (skeletons for content, spinners for actions, disabled controls during in-flight requests) — no dead buttons that silently do nothing.
- Admin analytics dashboard: charts for query volume, latency distribution, and error rate over time.

4.3 Frontend testing

- Unit tests for components and hooks (Vitest + React Testing Library).
- At least one end-to-end flow test (Playwright or Cypress) covering: login → upload → ask a question → get a grounded/refused answer.

5. Backend Requirements

5.1 API & auth

- REST API with explicit versioning (e.g. /api/v1/...).
- Auth: password hashing (bcrypt/argon2, correct salt/cost handling), JWT issuance and verification, refresh token rotation with revocation on logout/reuse detection.
- RBAC enforced via middleware/guards on every protected route, not scattered ad hoc checks.
- Rate limiting per user/IP on auth and query endpoints.

5.2 Document processing pipeline

- OCR stage for scanned PDFs/images (e.g. Tesseract), integrated so text-native and OCR'd documents flow into the same downstream chunking/embedding pipeline.
- Chunking and embedding logic built by you (as in Project 1), no RAG framework.
- Processing runs asynchronously via a job queue (RabbitMQ, BullMQ, or similar) — the upload endpoint should return immediately and processing should be observable via status polling or push updates.
- Idempotent, retryable jobs: a failed OCR or embedding step should retry with backoff and eventually surface a clear failure state, not hang forever.

5.3 Caching

- Redis (or similar) caching layer for: session/token validation, repeated-query result caching, and embedding cache to avoid recomputing embeddings for identical chunks.
- A documented cache invalidation strategy, what invalidates on document update/delete, and how staleness is bounded.

5.4 Data layer (MongoDB)

- Schema design for users, roles, documents, chunks (with vectors), chat threads/messages, and audit/analytics events.
- Indexing strategy justified (what's indexed, why, and what query patterns it serves).
- Vector search: the self-managed FAISS service defined in the required stack, running alongside MongoDB, with its own indexing and persistence strategy documented.

5.5 RAG + NLP bot

- Same grounding discipline as Project 1: retrieval-gated refusal (similarity threshold before generation, not just prompt instructions), citations, and explicit handling of empty/unreadable documents.
- Conversational memory: the bot should handle follow-up questions that reference prior turns in the same thread, without losing document grounding.

5.6 Backend testing

- Unit tests for core logic (chunking, retrieval, auth, RBAC checks).
- Integration tests hitting real endpoints against a test database.
- Load/latency testing (k6 or Locust) for the query endpoint, report p50/p95/p99 latency under concurrent load, and identify the bottleneck.

6. Infrastructure & Deployment

- Dockerfiles for each service (frontend, API, worker, OCR service if separate) and a docker-compose setup for local development spinning up the full stack (app services + MongoDB + Redis + queue).
- Kubernetes manifests (or a Helm chart) describing Deployments, Services, and Pods for each component, including resource requests/limits and a basic HorizontalPodAutoscaler for the API.
- Nginx as reverse proxy/ingress: TLS termination, routing to frontend/API, and basic rate limiting or request size limits at the edge.
- A written explanation of the DNS and networking model: how services discover each other inside the cluster, how ingress routes external traffic in, and how you'd handle this across environments (local vs. staging vs. prod), this can be documented and diagrammed rather than fully provisioned on real infrastructure, but it must be concrete, not hand-wavy.
- A CI outline (even if not fully wired to a real pipeline): what runs on every PR, lint, unit tests, build, and where integration/load tests would fit.

7. Analytics & Observability

- Structured logging across services (request IDs that trace a request end-to-end through queue/worker/API).
- An events pipeline capturing: queries asked, answers returned vs. refused, latency per stage (retrieval, generation, total), and errors.
- The analytics dashboard in the frontend should be reading real aggregated data, not mock numbers.

8. Non-Functional Requirements

Requirement	Target
Query latency	p95 under 3s for a cached/warm query; documented for cold path.
Concurrency	System should handle at least 20 concurrent users without falling over; document what breaks first beyond that.
Security	No plaintext secrets in repo, hashed passwords, server-enforced RBAC, tenant data isolation verified by test.
Resilience	A dependency failure (queue, cache, or OCR service down) degrades gracefully, documented behavior, not a crash.

9. Deliverables

- A GitHub repo with clean, logically staged commits, reviewable as a sequence of decisions, not a single dump.
- A working local demo runnable via docker-compose with setup instructions.
- Kubernetes manifests / Helm chart, even if only validated locally (kind/minikube) rather than on a real cluster.
- A test suite (frontend + backend) that actually runs, with a coverage summary.
- A load test report (script + results) for the query endpoint.
- A written design document (2–3 pages, longer than Project 1's is expected here) covering: architecture diagram, key tradeoffs, what you'd do differently with more time, and an honest list of what's stubbed or simplified and why.

10. Timeline (Deadline: 1 Week)

Day	Focus
Day 1	MongoDB schema, JWT auth (signup/login/refresh), password hashing, RBAC middleware, base API skeleton (Express/NestJS + FastAPI services wired together).
Day 2	Document upload endpoint, OCR integration (Tesseract), RabbitMQ/BullMQ job queue for async processing, processing-status tracking.
Day 3	Chunking + embeddings (sentence-transformers), self-managed FAISS service, retrieval with similarity-threshold gating, Redis caching layer.
Day 4	RAG/NLP bot: grounded prompt, citations, conversational memory, spaCy-based intent/entity layer; wire Ollama LLM calls end to end.
Day 5	Frontend: auth flows, RBAC-guarded routes, upload UX with progress/status, streaming chat UI, error/loading states, admin analytics dashboard.
Day 6	Testing (unit, integration, e2e, k6 load test), Dockerfiles + docker-compose, Kubernetes manifests/Helm chart, Nginx/ingress config.
Day 7	Observability (logging, Prometheus/Grafana), CI pipeline, bug fixes, load test report, design write-up, final pass on edge cases.

11. Constraints

- No LangChain/LlamaIndex/RAG frameworks.
- No third-party auth-as-a-service (e.g. no Auth0/Clerk/Firebase Auth), build the JWT/hashing/RBAC flow yourself.
- No substitutions to the required technology stack in Section 2 without prior sign-off.
- Ask questions to think out loud, not to get the answer handed to you, that's still welcome.

Sequencing matters as much as the code: get auth/RBAC and the core RAG loop solid first, then layer async processing, caching, and infrastructure on top.